

3.1 Análisis de Calidad con Sonar

Proyecto: ModuShield — Gateway de seguridad

1. Herramientas utilizadas

- SonarQube for IDE (extensión de Visual Studio Code) — utilizada durante el desarrollo para detección temprana de bugs, code smells y problemas de seguridad.
- SonarQube Community Build (v26.9.0.129388) — ejecutado localmente vía Docker, utilizado para el análisis global del proyecto y la generación del tablero de métricas.
- Apache Maven (a través del wrapper mvnw) con el plugin org.sonarsource.scanner.maven:sonar-maven-plugin para ejecutar el análisis sobre el proyecto multi-módulo.

El proyecto ModuShield es un proyecto Java (Spring Boot, Java 17) estructurado en Maven multi-módulo, compuesto por los módulos gateway-service y demo-api bajo un POM raíz común.

2. Métricas obtenidas

A continuación se documentan las métricas principales arrojadas por el análisis de SonarQube Community Build sobre el proyecto completo.

Métrica	Resultado
Bugs / Reliability (antes de la corrección)	2 hallazgos — calificación C
Bugs / Reliability (después de la corrección)	1 hallazgo — calificación mejorada
Vulnerabilities / Security	0 hallazgos — calificación A
Security Hotspots	0
Code Smells / Maintainability	9 hallazgos — calificación A
Coverage (cobertura de pruebas)	0.0% sobre 384 líneas a cubrir — no se integró reporte de JaCoCo
Duplicated Lines (duplicación)	5.8% sobre 1.3k líneas
Líneas de código analizadas	982 LOC

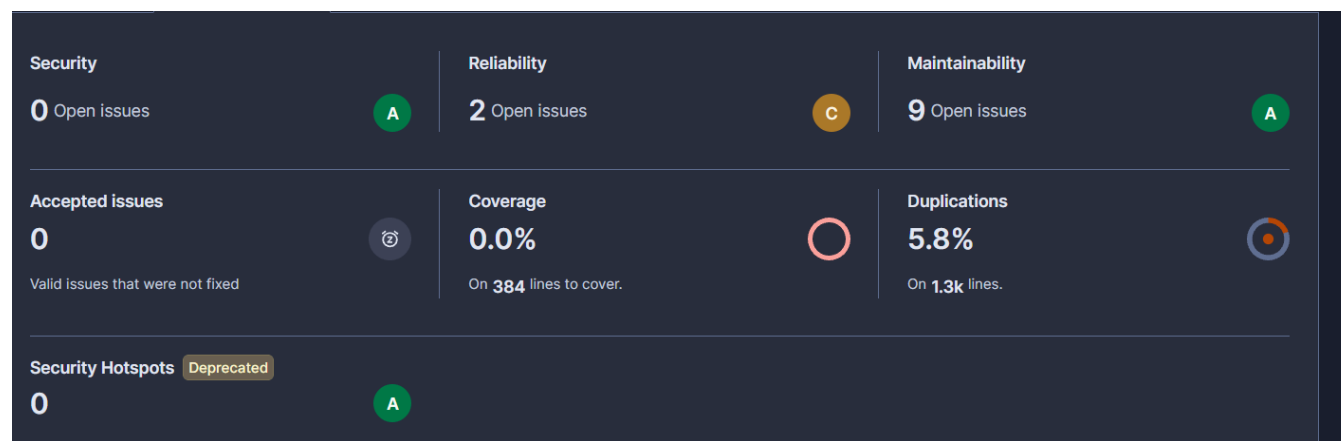


Figura 1. Tablero general del proyecto ModuShield en SonarQube.

3. Hallazgo seleccionado para revisión

Regla / Descripción: Se podría lanzar un "NullPointerException"; "getStatusCode()" puede devolver null

Categoría: Reliability (Bug) — Severidad Media

Ubicación: gateway-service/src/main/java/com/modushield/gateway/audit/AuditFilter.java, línea 41

Esfuerzo estimado por Sonar: 10 minutos

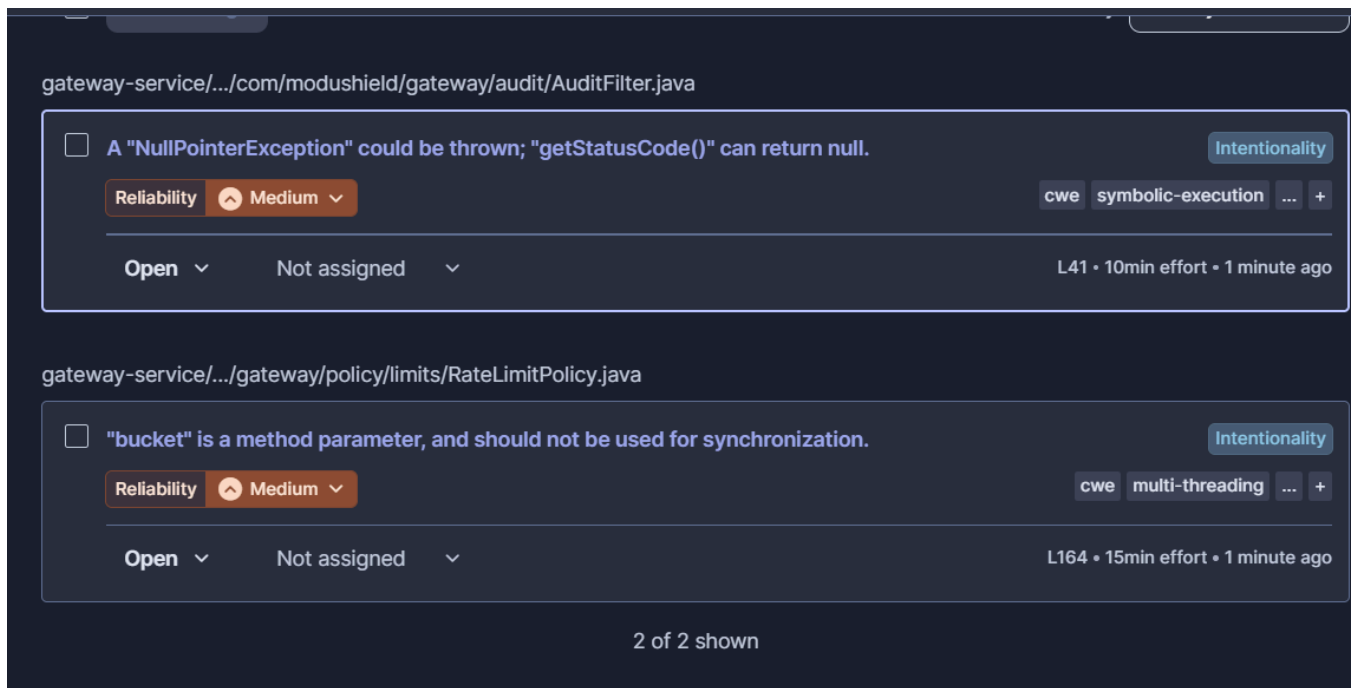


Figura 2. Hallazgos de Reliability detectados antes de la corrección (2 issues).

3.1 ¿Por qué el hallazgo era relevante?

El método `writeEvent()` invocaba `exchange.getResponse().getStatusCode()` dos veces por separado: una vez para validar si el resultado era null, y una segunda vez para obtener su valor:

```
int statusCode = exchange.getResponse().getStatusCode() == null
    ? 200
    : exchange.getResponse().getStatusCode().value();
```

El problema es que, al tratarse de un entorno reactivo (Spring WebFlux/Spring Cloud Gateway), el estado interno de la respuesta HTTP puede mutar de forma asíncrona entre una llamada y otra. Esto abre la posibilidad de una condición de carrera: que la primera llamada devuelva un valor no nulo (pasando la validación) pero que la segunda llamada devuelva null, provocando un `NullPointerException` real en producción durante el registro de auditoría.

Se determinó que este hallazgo NO era un falso positivo: aunque la probabilidad de que ocurra es baja, el patrón de doble invocación a un método que puede devolver null es una práctica insegura que SonarQube identifica correctamente mediante análisis simbólico (symbolic-execution).

4. Evidencia de la corrección

La corrección consistió en invocar `getStatusCode()` una única vez, almacenando el resultado en una variable local antes de evaluar su nulidad, garantizando una lectura consistente del valor:

```
var responseStatus = exchange.getResponse().getStatusCode();
int statusCode = responseStatus == null ? 200 : responseStatus.value();
```

Tras aplicar el cambio, se volvió a ejecutar el análisis completo con el comando:

```
.\mvnw.cmd clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
  "-Dsonar.projectKey=ModuShield" "-Dsonar.projectName=ModuShield" \
  "-Dsonar.host.url=http://localhost:9000" "-Dsonar.token=$env:SONAR_TOKEN"
```

El resultado fue BUILD SUCCESS, y al revisar nuevamente la sección de Reliability en SonarQube, el hallazgo sobre `AuditFilter.java` ya no aparece en la lista, quedando únicamente el hallazgo correspondiente a `RateLimitPolicy.java` (no seleccionado para esta revisión).

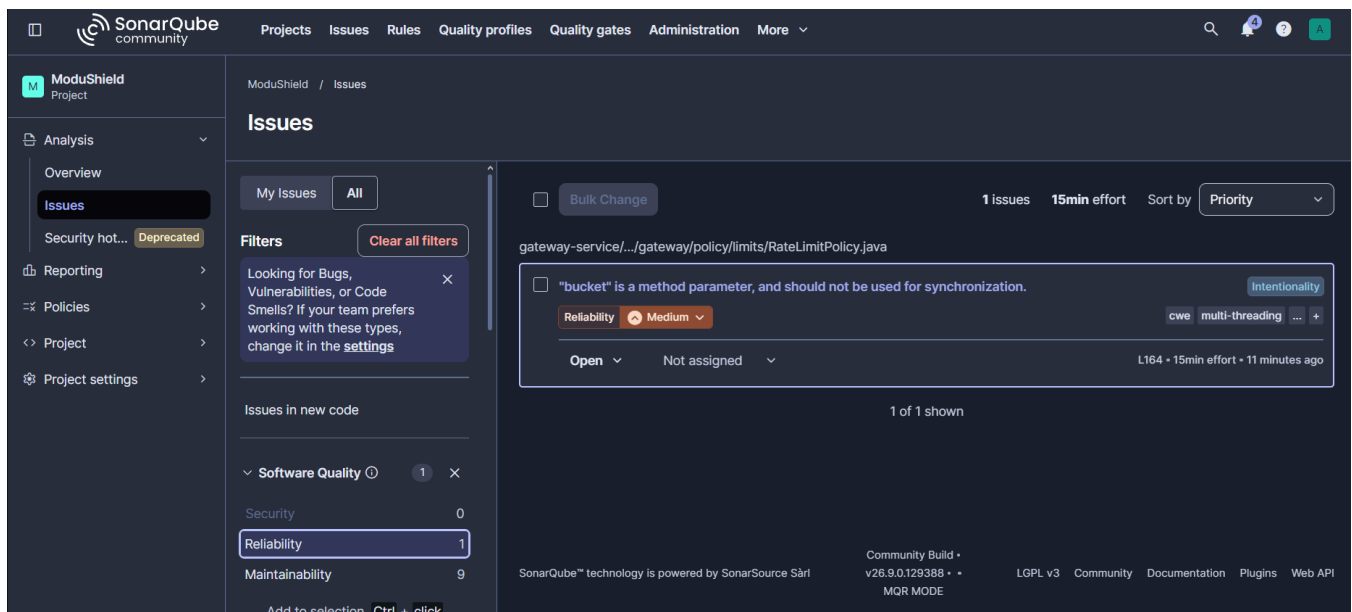


Figura 3. Reliability tras la corrección: el hallazgo de AuditFilter.java fue resuelto (1 issue restante, correspondiente a un archivo distinto).

5. Conclusión

El análisis con SonarQube Community Build permitió identificar un bug real de confiabilidad relacionado con el manejo inseguro de valores potencialmente nulos en un entorno reactivo. La corrección aplicada —consolidar la llamada al método en una única variable local— eliminó el riesgo de condición de carrera sin alterar el comportamiento funcional del filtro de auditoría, y quedó verificada mediante una segunda ejecución del análisis que confirma la desaparición del hallazgo.